

RNN Application

- Seq2Seq -

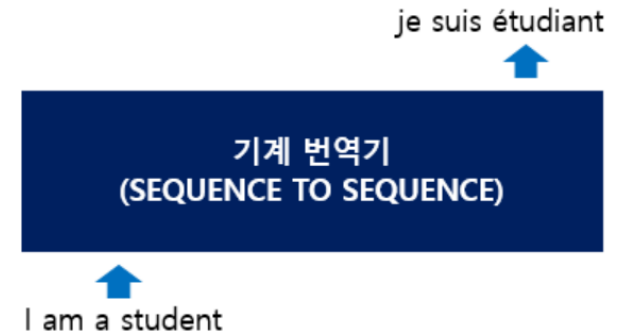
Fall 2025

School of IT Convergence

Prof. Dongsup Jin

Seq2Seq (Sequence to Sequence)

- Seq2Seq
 - Translate a sentence into another sentence
 - Input : sequence
 - Output: sequence
 - Dataset
 - Type: parallel corpora
 - A corpora of texts translated into two or more languages
 - Pair: Original sentence – Translated sentence



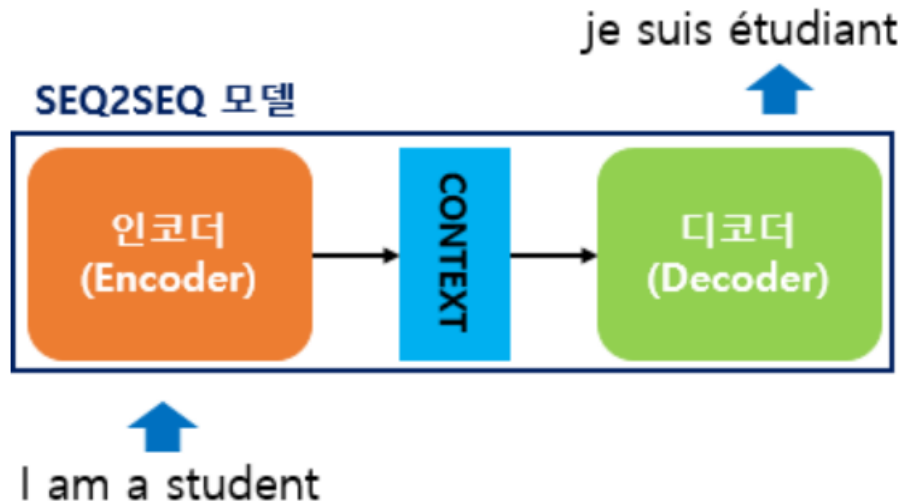
(Ex1) I like programming – 나는 프로그래밍을 좋아한다

(Ex2) Much of personal computing is about "can you top this?"

- 개인용 컴퓨터 사용의 상당 부분은 "이것보다 뛰어날 수 있느냐"

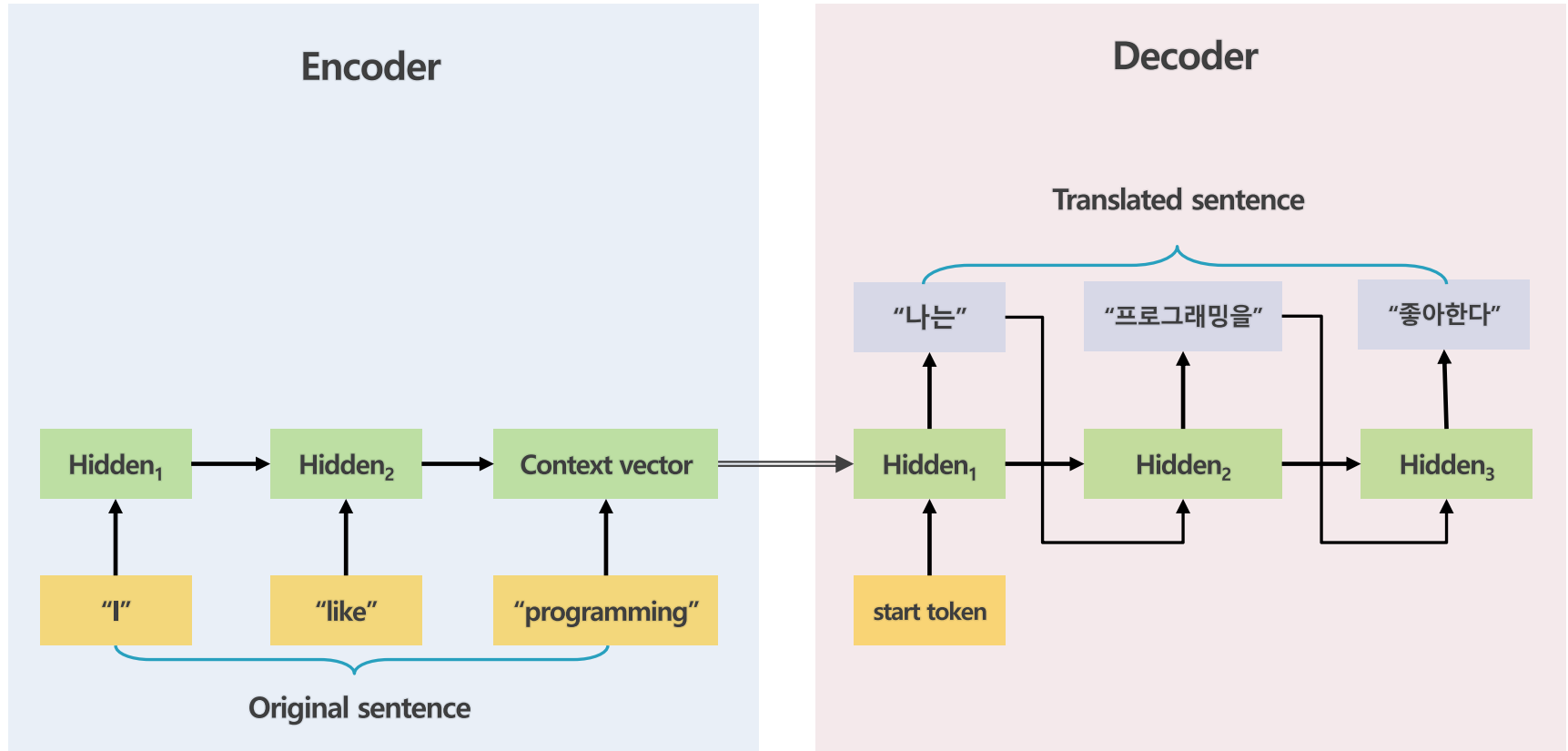
Simple Seq2Seq (1)

- Simple Seq2Seq
 - Translate an alphabet sequence into another alphabet sentence
 - (Ex) English “hello” → Spanish “hola”
 - Model structure
 - Two RNNs: Encoder and Decoder



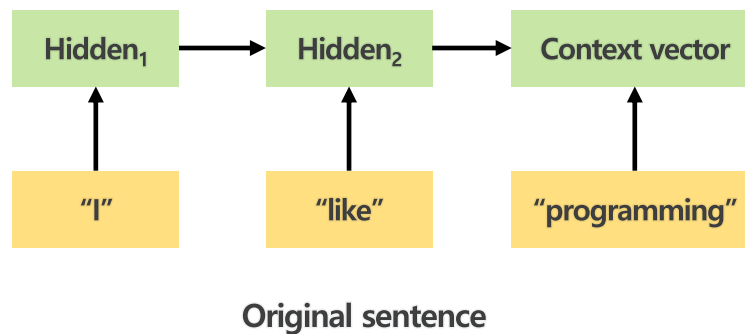
Simple Seq2Seq (2)

- Simple Seq2Seq translation model



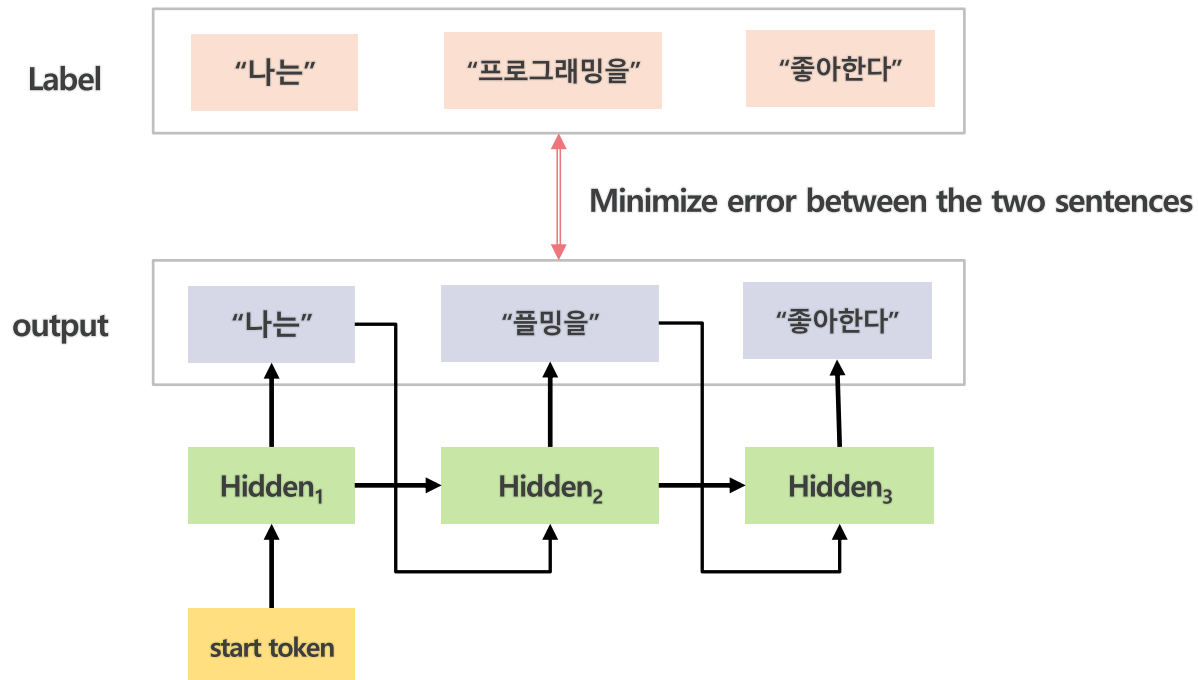
Simple Seq2Seq (3)

- Encoder
 - A RNN that learns the contents of the original sentence (or sequence)
 - Take all words in the original sentence
 - Create a single fixed-size tensor containing the meaning of the sentence
 - Context vector
 - Condense the meaning and content of the original sentence



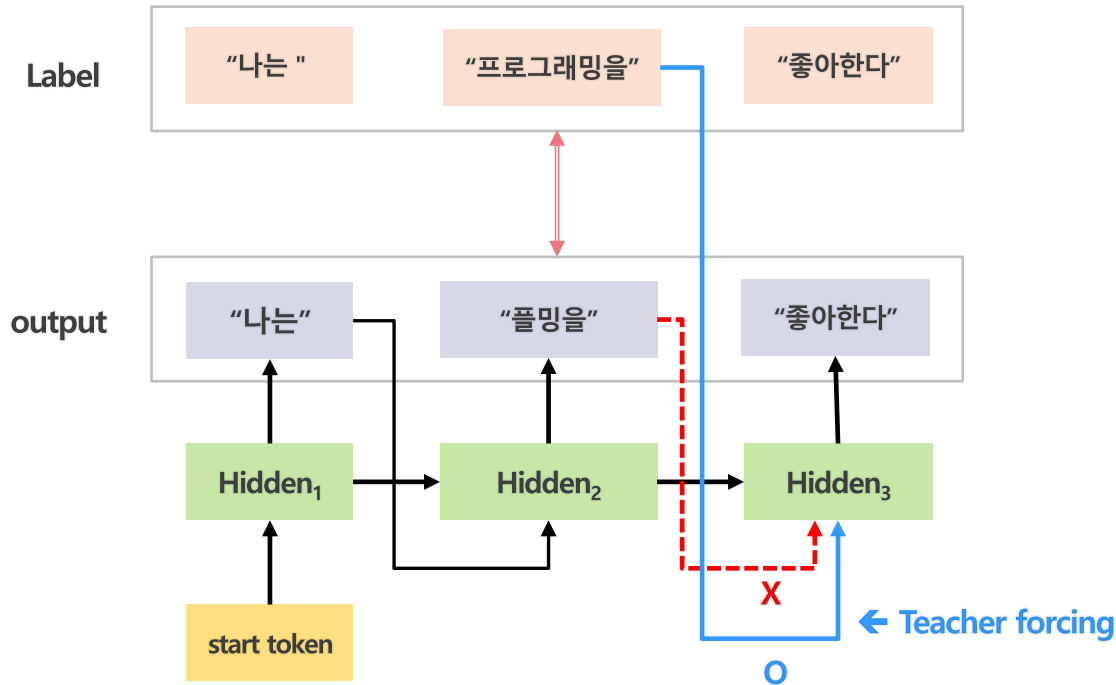
Simple Seq2Seq (4)

- Decoder
 - A RNN that predict the tokens in the translation from the context vector
 - Predict the output tokens in order
 - Train a model that minimizes the error between the predicted tokens and the real label tokens



Simple Seq2Seq (5)

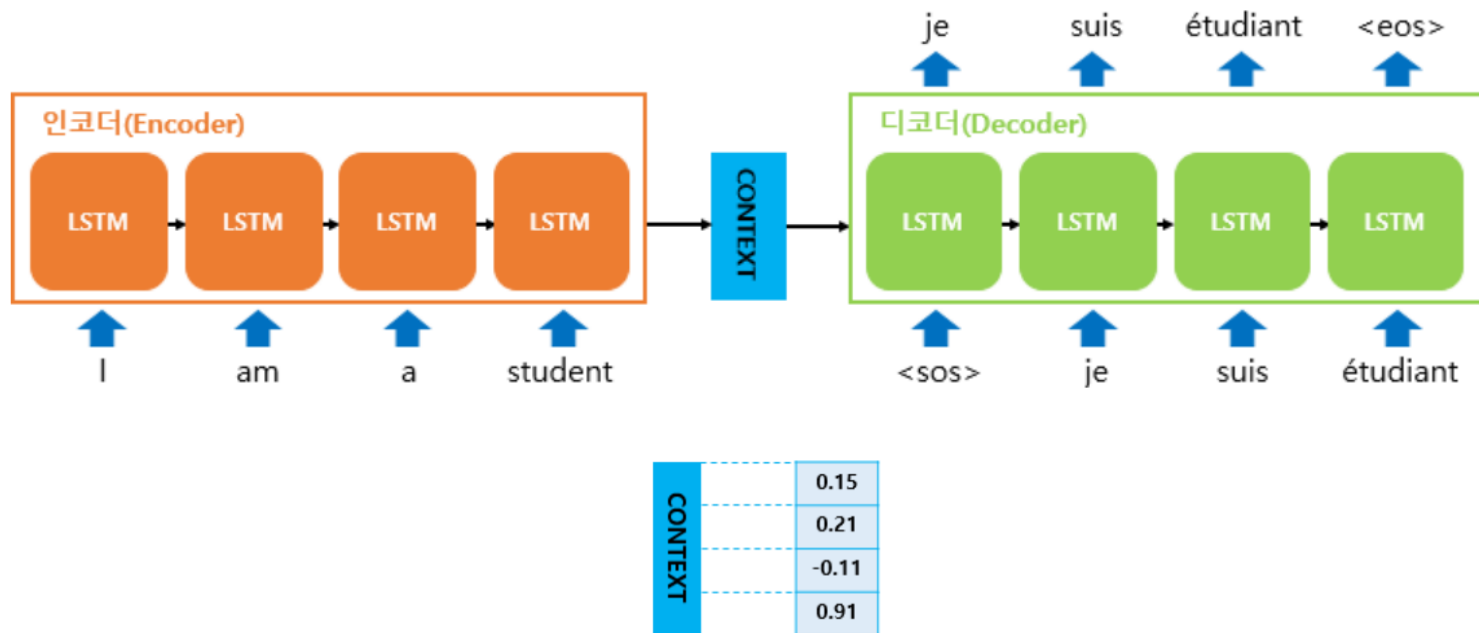
- Teacher forcing
 - Input **the real label vector** into the current decoder hidden neuron, **instead of the output vector** in the previous decoder hidden neuron



Simple Seq2Seq (6)

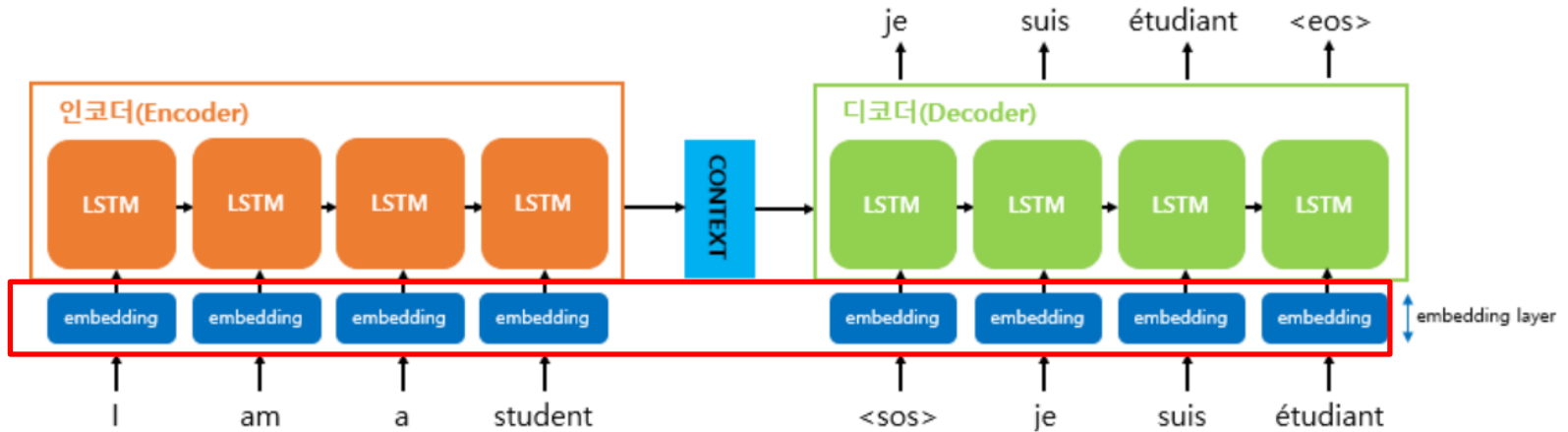
- Example

- (English) I am a student
- (France) Je suis étudiant.



Simple Seq2Seq (7)

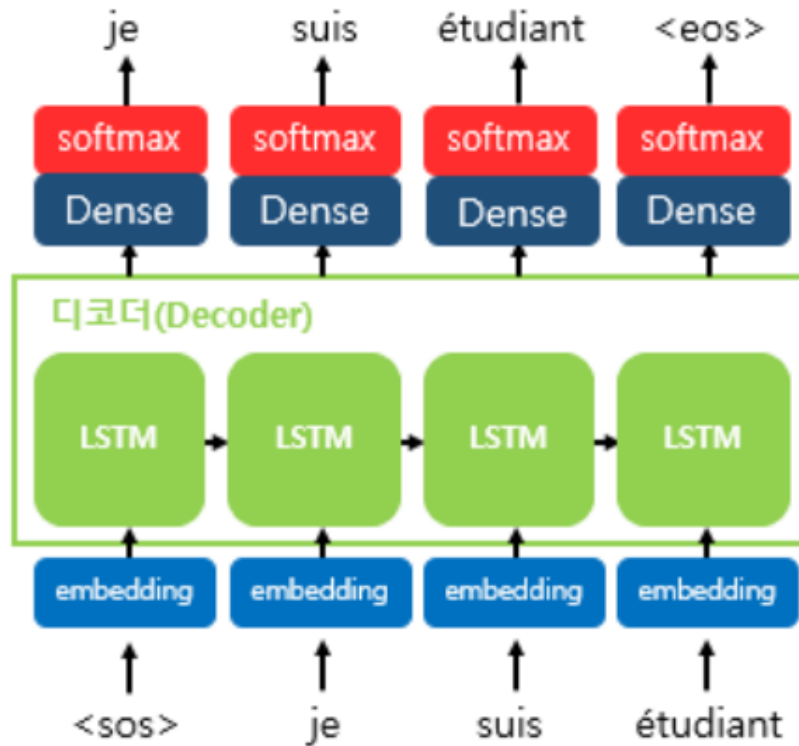
- Example



I	0.157	am	0.78	a	0.75	student	0.88
	-0.25		0.29		-0.81		-0.17
	0.478		-0.96		0.96		0.29
	-0.78		0.52		0.12		0.48

Simple Seq2Seq (8)

- Example



Simple Seq2Seq (9)

- (Ex) Dataset

- (France-English parallel corpora) <http://www.manythings.org/anki>
- (Source) <https://blog.keras.io/a-ten-minute-introduction-to-sequence-to-sequence-learning-in-keras.html>

	src	tar
14508	I refused to pay.	Je refusai de payer.
44605	Do you speak Esperanto?	Parlez-vous espéranto ?
48706	This tape isn't sticky.	Ce ruban adhésif ne colle pas.
55442	Tonight would be better.	Ce soir serait mieux.
5633	I'll call you.	Je vous appellerai.
39097	He proved to be a spy.	Il s'est révélé être un espion.
29026	I'm indebted to you.	J'ai une dette envers vous.
32825	Have it your own way.	Faites ce qui vous plaira.
27062	Did you read it all?	L'avez-vous lue en entier ?
1094	Choose one.	Choisis-en une.

	src	tar
59698	I'll take responsibility.	¶t J'assumerai la responsabilité. ¶n
49350	We're happy to be here.	¶t Nous sommes heureux d'être ici. ¶n
56905	Bears are very dangerous.	¶t Les ours sont très dangereux. ¶n
39277	How big is your house?	¶t De quelle taille est ta maison ? ¶n
55564	We have come a long way.	¶t Nous avons fait un long voyage. ¶n
1122	Don't move!	¶t Ne bouge pas ! ¶n
17884	He is intelligent.	¶t Il est intelligent. ¶n
58	No way!	¶t Sans façons ! ¶n
20241	The motor stopped.	¶t Le moteur s'est arrêté. ¶n
56880	Are you the group leader?	¶t Es-tu la meneuse du groupe ? ¶n

Simple Seq2Seq in Pytorch (1)

1. Import several packages and modules

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import random
import matplotlib.pyplot as plt
```

2. Set hyper parameters

```
vocab_size = 256 # 총 아스키 코드 개수
x_ = list(map(ord, "hello")) # 아스키 코드 리스트로 변환
y_ = list(map(ord, "hola")) # 아스키 코드 리스트로 변환

print("hello -> ", x_)
print("hola -> ", y_)

x = torch.LongTensor(x_)
y = torch.LongTensor(y_)
```

```
hello -> [104, 101, 108, 108, 111]
hola -> [104, 111, 108, 97]
```

Word embedding !!

"hello" $\leftrightarrow$ "hola"

Simple Seq2Seq in Pytorch (2)

3. Define Seq2Seq model

```
class Seq2Seq(nn.Module):
    def __init__(self, vocab_size, hidden_size):
        super(Seq2Seq, self).__init__()
        self.n_layers = 1
        self.hidden_size = hidden_size
        self.embedding = nn.Embedding(vocab_size, hidden_size)
        self.encoder = nn.GRU(hidden_size, hidden_size)
        self.decoder = nn.GRU(hidden_size, hidden_size)
        self.project = nn.Linear(hidden_size, vocab_size)

    def forward(self, inputs, targets):
        # 인코더에 들어갈 입력
        initial_state = self._init_state()
        embedding = self.embedding(inputs).unsqueeze(1)
        # embedding = [seq_len, batch_size, embedding_size]

        # 인코더 (Encoder)
        encoder_output, encoder_state = self.encoder(embedding, initial_state)
        # encoder_output = [seq_len, batch_size, hidden_size]
        # encoder_state = [n_layers, seq_len, hidden_size]
```

Simple Seq2Seq in Pytorch (3)

3. Define Seq2Seq model (2)

```
# 디코더에 들어갈 입력
decoder_state = encoder_state
decoder_input = torch.LongTensor([0])

# 디코더 (Decoder)
outputs = []
for i in range(targets.size()[0]):
    decoder_input = self.embedding(decoder_input).unsqueeze(1)
    decoder_output, decoder_state = self.decoder(decoder_input, decoder_state)
    projection = self.project(decoder_output)
    outputs.append(projection)

    #티처 포싱(Teacher Forcing) 사용
    decoder_input = torch.LongTensor([targets[i]]) → Input label data, not decoder_output

outputs = torch.stack(outputs).squeeze()
return outputs

def _init_state(self, batch_size=1):
    weight = next(self.parameters()).data
    return weight.new(self.n_layers, batch_size, self.hidden_size).zero_()
```

Simple Seq2Seq in Pytorch (4)

4. Declare a Seq2Seq model

```
seq2seq = Seq2Seq(vocab_size, 16)
```

```
Seq2Seq(  
  (embedding): Embedding(256, 16)  
  (encoder): GRU(16, 16)  
  (decoder): GRU(16, 16)  
  (project): Linear(in_features=16, out_features=256, bias=True)  
)
```

5. Set optimization

```
criterion = nn.CrossEntropyLoss()  
optimizer = torch.optim.Adam(seq2seq.parameters(), lr=1e-3)
```

6. Train a module

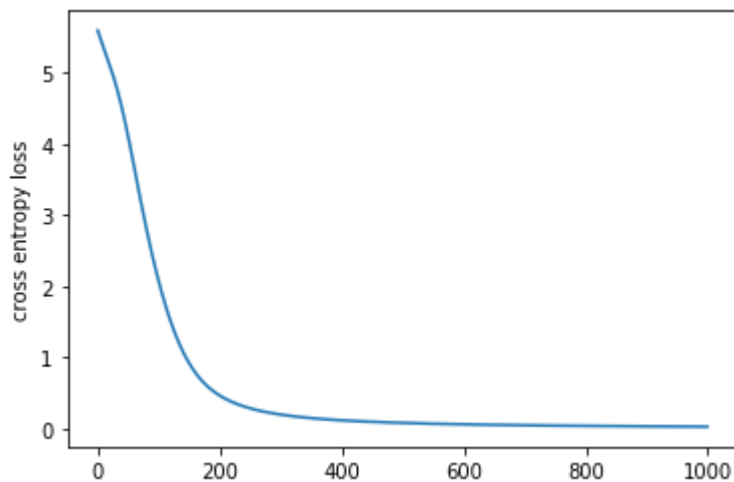
```
log = []  
for i in range(1000):  
    prediction = seq2seq(x, y)  
    loss = criterion(prediction, y)  
    optimizer.zero_grad()  
    loss.backward()  
    optimizer.step()  
    loss_val = loss.data  
    log.append(loss_val)  
    if i % 100 == 0:  
        print("\n 반복:%d 오차: %s" % (i, loss_val.item()))  
        _, top1 = prediction.data.topk(1, 1)  
        print([chr(c) for c in top1.squeeze().numpy().tolist()])
```

```
반복:0 오차: 5.596976280212402  
['g', 'l', 'x98', 'l']  
  
반복:100 오차: 2.069061756134033  
['h', 'o', 'l', 'a']  
  
반복:200 오차: 0.4633035957813263  
['h', 'o', 'l', 'a']  
  
반복:300 오차: 0.19558477401733398  
['h', 'o', 'l', 'a']  
  
반복:400 오차: 0.11498594284057617  
['h', 'o', 'l', 'a']  
  
반복:500 오차: 0.07863441854715347  
['h', 'o', 'l', 'a']  
  
반복:600 오차: 0.058321841061115265  
['h', 'o', 'l', 'a']  
  
반복:700 오차: 0.04549476131796837  
['h', 'o', 'l', 'a']  
  
반복:800 오차: 0.03673341125249863  
['h', 'o', 'l', 'a']  
  
반복:900 오차: 0.030412672087550163  
['h', 'o', 'l', 'a']
```

Simple Seq2Seq in Pytorch (5)

7. Plot error

```
plt.plot(log)  
plt.ylabel('cross entropy loss')  
plt.show()
```



Seq2Seq in Youtube

- Seq2Seq + attention model - <https://youtu.be/WsQLdu2JMgl>

시퀀스 투 시퀀스 + 어텐션

인공신경망 기계번역

Q & A
